

SKPLOO-03

SPESIFIKASI KEBUTUHAN PERANGKAT LUNAK

Jurnal Kita (JuKi)

untuk:

Grace Evelyn Simon

Dipersiapkan oleh:

K02

Nadine Arindy Octavia - 18224012

Danya Damaris - 18224024

Anisa Aulia Alhaqi - 18224080

Zahrah Nur Azizah - 18224092

Riko Satriya Giovanni - 18224108

Program Studi Sistem dan Teknologi Informasi

STEI - ITB

Jl. Ganesha 10, Bandung 40132

	Program Studi Sistem dan Teknologi Informasi STEI – ITB	Nomor Dokumen		Halaman
		<i>SKPLOO-03</i>		<i>1/42</i>
		Revisi	<i>A</i>	<i>Tgl: 01/04/2026</i>

Daftar Perubahan

Revisi	Deskripsi
A	Menyesuaikan kelas diagram berdasarkan revisi pada asistensi, yaitu memastikan adanya 3 jenis kelas pada setiap use case
B	
C	
D	
E	
F	
G	
H	

Daftar Isi

1 Pendahuluan	7
1.1 Tujuan Penulisan Dokumen.	7
1.2 Lingkup Masalah	7
1.3 Definisi, Istilah, dan Singkatan	7
1.4 Aturan Penomoran	8
1.5 Referensi	8
1.6 Deskripsi Umum Dokumen (Ikhtisar)	8
2 Deskripsi Umum Perangkat Lunak	9
2.1 Deskripsi Umum Sistem	9
2.2 Deskripsi Umum Perangkat Lunak	10
2.3 Karakteristik Pengguna	11
2.4 Batasan Perangkat Lunak	11
2.5 Lingkungan Operasi Perangkat Lunak	12
3 Deskripsi Kebutuhan Perangkat Lunak	13
3.1 Kebutuhan Fungsional	13
3.2 Kebutuhan Non Fungsional	14
4 Model Analisis	16
4.1 Model Use Case	16
4.1.1 Identifikasi Actor	16
4.1.2 Identifikasi Use Case	16
4.1.3 Diagram Use Case	17
4.1.4 Skenario Use Case	17
4.1.4.1 Skenario Use Case 01	17
4.1.4.2 Skenario Use Case 02	18
4.1.4.3 Skenario Use Case 03	19
4.1.4.4 Skenario Use Case 04	19
4.1.4.5 Skenario Use Case 05	20
4.1.4.6 Skenario Use Case 06	20
5 Diagram Kelas	22
5.1 Use Case Mencatat Entri Harian	22
5.1.1 Identifikasi Kelas Use Case Mencatat Entri Harian	22
5.1.2 Identifikasi Kelas Use Case Mencatat Entri Harian	23
5.2 Use Case Melihat Hasil Visualisasi dari Data Pengguna	25
5.2.1 Identifikasi Kelas Use Case Melihat Hasil Visualisasi dari Data Pengguna	25

5.2.2 Identifikasi Kelas Use Case Melihat Hasil Visualisasi dari Data Pengguna	25
5.3 Use Case Menghapus Entri Data	27
5.3.1 Identifikasi Kelas Use Case Menghapus Entri Data	27
5.3.2 Identifikasi Kelas Use Case Menghapus Entri Data	28
5.4 Use Case Memperbaharui Entri Data	30
5.4.1 Identifikasi Kelas Use Case Memperbaharui Entri Data	30
5.4.2 Identifikasi Kelas Use Case Memperbaharui Entri Data	30
5.5 Use Case Meninjau Entri Data yang Telah Dimasukkan	32
5.5.1 Identifikasi Kelas Use Case Meninjau Entri Data yang Telah Dimasukkan	32
5.5.2 Identifikasi Kelas Use Case Meninjau Entri Data yang Telah Dimasukkan	33
5.6 Use Case Melakukan Pencarian dan Filtrasi pada Entri Jurnal	35
5.6.1 Identifikasi Kelas Use Case Melakukan Pencarian dan Filtrasi pada Entri Jurnal	35
5.6.2 Identifikasi Kelas Use Case Melakukan Pencarian dan Filtrasi pada Entri Jurnal	36
5.7 Diagram Kelas Keseluruhan	38
6 Keruntutan (Traceability)	41
6.1.1 Kebutuhan Fungsional vs Use Case	41
6.1.2 Kebutuhan Fungsional vs Use Case vs Diagram Kelas	41

Daftar Tabel

<i>Tabel 1.3. Definisi, Istilah, dan Singkatan</i>	7
<i>Tabel 1.4. Aturan Penomoran</i>	8
<i>Tabel 2.1.1 Deskripsi proses dan peran sistem</i>	10
<i>Tabel 2.3.1. Karakteristik Pengguna</i>	11
<i>Tabel 3.1.1. Kebutuhan Fungsional</i>	13
<i>Tabel 3.2.1. Tabel Kebutuhan Non Fungsional</i>	14
<i>Tabel 4.1.1.1. Identifikasi Actor</i>	16
<i>Tabel 4.1.2.1. Identifikasi Use Case</i>	16
<i>Tabel 4.1.4.1.1 Skenario Use Case 01</i>	17
<i>Tabel 4.1.4.1.2 Skenario Use Case 02</i>	18
<i>Tabel 4.1.4.1.3 Skenario Use Case 03</i>	19
<i>Tabel 4.1.4.1.4 Skenario Use Case 04</i>	19
<i>Tabel 4.1.4.1.5 Skenario Use Case 05</i>	20
<i>Tabel 4.1.4.1.6 Skenario Use Case 06</i>	20
<i>Tabel 5.1.1. Daftar Kelas Use Case Mencatat Entri Harian</i>	22
<i>Tabel 5.1.2. Identifikasi Kelas Use Case Mencatat Entri Harian</i>	23
<i>Tabel 5.2.1. Daftar Kelas Use Case Melihat Hasil Visualisasi dari Data Pengguna</i>	25
<i>Tabel 5.2.2. Identifikasi Kelas Use Case Melihat Hasil Visualisasi dari Data Pengguna</i>	26
<i>Tabel 5.3.1. Daftar Kelas Use Case Menghapus Entri Data</i>	27
<i>Tabel 5.3.2. Spesifikasi Kelas Use Case Menghapus Entri Data</i>	28
<i>Tabel 5.4.1. Daftar Kelas Use Case Memperbaharui Entri Data</i>	30
<i>Tabel 5.4.2. Spesifikasi Kelas Use Case Memperbaharui Entri Data</i>	31
<i>Tabel 5.5.1. Daftar Kelas Berdasarkan Use Case Meninjau Entri Data yang Telah Dimasukkan</i>	33
<i>Tabel 5.5.2. Spesifikasi Kelas Use Case Meninjau Entri Data yang Telah Dimasukkan</i>	33
<i>Tabel 5.6.1. Daftar Kelas Use Case Melakukan Pencarian dan Filtrasi pada Entri Jurnal</i>	35
<i>Tabel 5.6.2. Spesifikasi Kelas Use Case Melakukan Pencarian dan Filtrasi pada Entri Jurnal</i>	36
<i>Tabel 5.7. Spesifikasi Kelas Keseluruhan</i>	38
<i>Tabel 6.1.1. Kebutuhan Fungsional vs Use Case</i>	41
<i>Tabel 6.1.2. Kebutuhan Fungsional vs Use Case vs Diagram Kelas</i>	41

Daftar Gambar

<i>Gambar 2.1 Swimlane Diagram Sistem</i>	9
<i>Gambar 2.2 Diagram Konteks</i>	11
<i>Gambar 4.1 Diagram Use Case</i>	17
<i>Gambar 5.1 Use Case Mencatat Entri Harian</i>	23
<i>Gambar 5.2 Use Case Melihat Hasil Visualisasi dari Data Pengguna</i>	26
<i>Gambar 5.3. Class Diagram Menghapus Entri Data</i>	28
<i>Gambar 5.4. Class Diagram Memperbaharui Entri Data</i>	31
<i>Gambar 5.5 Meninjau Entri Data yang Telah Dimasukkan</i>	33
<i>Gambar 5.6 Melakukan Pencarian dan filtrasi pada Entri Jurnal</i>	36
<i>Gambar 5.7 Diagram Kelas Keseluruhan</i>	38

1 Pendahuluan

1.1 Tujuan Penulisan Dokumen.

Dokumen Spesifikasi Kebutuhan Perangkat Lunak (SKPL) ini disusun untuk mendefinisikan secara sistematis kebutuhan sistem **JuKi**, yaitu aplikasi pemantauan suasana hati dan aktivitas self-care berbasis digital. Dokumen ini bertujuan untuk mengidentifikasi dan mendeskripsikan kebutuhan fungsional serta non-fungsional perangkat lunak secara terstruktur sebagai landasan dalam proses perancangan, pengembangan, pengujian, dan evaluasi sistem.

Dokumen ini digunakan oleh pihak-pihak yang terlibat dalam pengembangan dan pengelolaan sistem, termasuk analis sistem yang merumuskan kebutuhan, pengembang perangkat lunak (*programmer*) yang mengimplementasikan sistem, penguji (*tester*) yang melakukan verifikasi dan validasi, manajer proyek yang mengoordinasikan pengembangan, serta administrator sistem yang mengelola operasional aplikasi. SKPL ini berfungsi sebagai pedoman resmi untuk memastikan bahwa sistem dikembangkan sesuai dengan kebutuhan yang telah ditetapkan, termasuk dukungan terhadap fitur bagi Pengguna Umum, Pengguna Terdaftar, dan Administrator Sistem, serta tidak melampaui batasan fungsi yang telah didefinisikan.

1.2 Lingkup Masalah

Kesadaran terhadap pentingnya kesehatan mental terus meningkat, namun masih banyak orang yang belum memiliki alat digital yang sederhana dan terstruktur untuk memantau kondisi emosional mereka secara konsisten. Proses refleksi diri sering dilakukan secara tidak sistematis, sehingga pengguna mengalami kesulitan dalam mengidentifikasi pola suasana hati maupun kebiasaan yang mempengaruhi kesejahteraan mental mereka dalam jangka waktu tertentu. **Perangkat lunak JuKi** dirancang sebagai aplikasi yang memungkinkan pengguna mencatat suasana hati harian berdasarkan kategori emosi tertentu, menambahkan jurnal atau refleksi pribadi dalam bentuk teks, mencatat target self-care atau aktivitas positif, serta memantau progres kebiasaan tersebut. Sistem juga menyediakan fitur visualisasi pola emosional dalam periode mingguan atau bulanan melalui grafik atau ringkasan statistik, serta mendukung pencarian entri jurnal berdasarkan tanggal atau kata kunci. Aplikasi ini dilengkapi dengan fitur manajemen akun pengguna untuk mendukung keamanan dan sinkronisasi data. Perangkat lunak ini berfungsi sebagai alat bantu refleksi diri dan pemantauan kebiasaan, serta tidak memberikan interpretasi medis, diagnosis, maupun rekomendasi klinis, dengan tetap menempatkan aspek privasi dan keamanan data sebagai bagian penting dalam pengembangannya.

1.3 Definisi, Istilah, dan Singkatan

Tabel 1.3. Definisi, Istilah, dan Singkatan

Singkatan, Akronim, atau Istilah	Penjelasan
P/L	Singkatan dari Perangkat Lunak yang merupakan sekumpulan data elektronik, instruksi, atau program yang tersimpan di dalam komputer.
SKPL	Singkatan dari Spesifikasi Kebutuhan Perangkat Lunak yang merupakan dokumen formal yang merinci fungsionalitas, batasan, dan antarmuka sistem yang akan dikembangkan

1.4 Aturan Penomoran

Tabel 1.4. Aturan Penomoran

Hal/Bagian	Penomoran	Keterangan
Aktivitas	AKXX	Digunakan untuk mengidentifikasi setiap aktivitas dalam proses bisnis atau activity diagram.
Kebutuhan Fungsional	FXX	Digunakan untuk mengidentifikasi kebutuhan fungsional sistem yang harus dipenuhi oleh perangkat lunak.
Kebutuhan Non-Fungsional	NFXX	Digunakan untuk mengidentifikasi kebutuhan non-fungsional seperti keamanan, performa, dan keandalan sistem.
Aktor	AXX	Digunakan untuk mengidentifikasi aktor yang berinteraksi dengan sistem dalam use case diagram.
Use Case	UCXX	Digunakan untuk mengidentifikasi skenario interaksi antara aktor dan sistem.

1.5 Referensi

Referensi utama yang digunakan untuk perangkaian SKPL ini adalah buku Software Engineering (Edisi ke-10) karya Ian Sommerville. Hal itu didasarkan pada kedalaman materi yang komprehensif mengenai topik Dasar Rekayasa Perangkat Lunak.

1.6 Deskripsi Umum Dokumen (Ikhtisar)

Dokumen ini memuat spesifikasi kebutuhan perangkat lunak **JuKi**. Pada bagian Pendahuluan, dokumen menjelaskan tujuan pembuatan dokumen, lingkup masalah yang dibahas, serta standar atau istilah yang digunakan di dalam dokumen. Selain itu, bagian ini juga mencantumkan berbagai referensi yang menjadi dasar pertimbangan dalam proses perancangan perangkat lunak.

Pada bagian Deskripsi Umum Perangkat Lunak, dijelaskan gambaran umum aplikasi, termasuk bagaimana interaksi antara pengguna dengan sistem serta proses penggunaan aplikasi secara keseluruhan. Bagian ini juga membahas karakteristik pengguna yang akan menggunakan aplikasi, serta menjelaskan batasan-batasan sistem dan lingkungan operasi tempat aplikasi akan dijalankan.

Pada bagian Deskripsi Kebutuhan Perangkat Lunak, dijelaskan secara rinci kebutuhan yang harus dipenuhi oleh perangkat lunak yang akan dikembangkan. Kebutuhan tersebut mencakup kebutuhan fungsional, yaitu fungsi atau layanan yang harus disediakan oleh sistem, serta kebutuhan non fungsional, yaitu aspek kualitas seperti kinerja, keamanan, dan keandalan yang harus dimiliki oleh perangkat lunak.

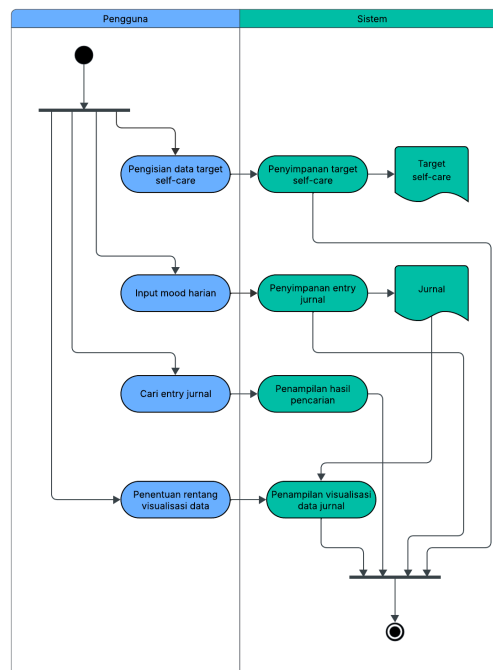
2 Deskripsi Umum Perangkat Lunak

2.1 Deskripsi Umum Sistem

Sistem yang dirancang akan memantau kesehatan mental pengguna dan menyediakan alat refleksi diri terstruktur dan konsisten. Sistem ini memungkinkan pengguna mencatat suasana hati harian berdasarkan kategori tertentu, menambahkan refleksi atau jurnal pribadi, serta menyimpan data secara terorganisir. Selain itu, sistem mendukung pencatatan target self-care atau aktivitas positif sehingga pengguna dapat memantau perkembangan kebiasaan yang berkontribusi pada kesejahteraan mental.

Dalam pengolahan data, sistem menyediakan fitur ringkasan statistik dan visualisasi tren suasana hati dalam periode mingguan maupun bulanan. Bentuk visualisasi dapat berupa grafik garis, kalender emosi, atau ringkasan numerik sederhana untuk membantu pengguna mengidentifikasi pola emosional. Sistem juga dilengkapi dengan fitur pencarian entri jurnal berdasarkan tanggal maupun kata kunci guna memudahkan akses terhadap catatan sebelumnya.

Sistem memperhatikan privasi dan keamanan data karena informasi yang disimpan bersifat personal. Oleh karena itu, diperlukan mekanisme perlindungan data yang memadai. Sistem ini dirancang sebagai alat pendukung refleksi diri dan bukan sebagai layanan diagnosis atau interpretasi medis, sehingga batasan fungsi tersebut perlu didefinisikan secara jelas dalam tahap requirement gathering.



Gambar 2.1 Swimlane Diagram Sistem

Tabel 2.1.1 Deskripsi proses dan peran sistem

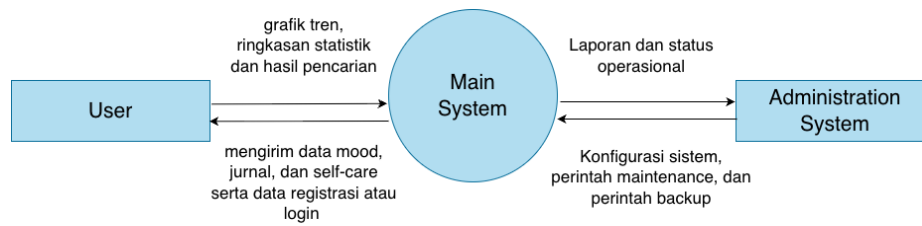
Proses	Deskripsi Proses	Peran Sistem
Pengguna mengisi data target <i>self-care</i>	Pengguna mencatat target aktivitas positif yang ingin mereka lakukan sebagai bentuk pemeliharaan akan diri sendiri. Data mengenai target ini mengandung jenis aktivitas yang dilakukan dan frekuensi/durasi aktivitas tersebut.	Sistem menerima input dari pengguna dan menyimpannya dalam database.
Pengguna menginput konten jurnal	Pengguna mencatat jurnal yang berisi tingkat suasana hati, judul, deskripsi, foto, pemicu, tanggal, dan waktu.	Sistem menerima input dari pengguna dan menyimpannya dalam database.
Pengguna mencari jurnal	Pengguna mencari jurnal berdasarkan kata kunci.	Sistem melakukan <i>searching</i> dan menampilkan jurnal berdasarkan kata kunci yang diinput pengguna.
Pengguna melihat visualisasi data	Pengguna melihat visualisasi data suasana hati dan pemicu.	Sistem menampilkan visualisasi data suasana hati dan pemicu.

2.2 Deskripsi Umum Perangkat Lunak

Perangkat lunak JuKi merupakan aplikasi desktop yang dirancang untuk mendukung pencatatan, pengelolaan, dan penyajian data suasana hati serta aktivitas self-care pengguna secara terstruktur. Aplikasi ini memungkinkan pengguna untuk mencatat kondisi emosional harian, menuliskan jurnal pribadi, serta mencatat target aktivitas self-care yang dilakukan sehari-hari. Data yang dimasukkan oleh pengguna disimpan secara lokal pada perangkat dan diolah oleh sistem untuk menghasilkan informasi berupa grafik tren suasana hati, ringkasan statistik, serta hasil pencarian data, sehingga pengguna dapat memahami pola emosional dan kebiasaan self-care mereka dari waktu ke waktu.

Dalam operasionalnya, sistem JuKi berinteraksi langsung dengan pengguna sebagai aktor utama. Pengguna memasukkan data suasana hati, jurnal, dan aktivitas self-care ke dalam sistem, kemudian sistem menyimpan dan mengolah data tersebut secara lokal untuk ditampilkan kembali dalam bentuk

riwayat data, hasil pencarian, maupun visualisasi sederhana. Aplikasi ini dirancang sebagai alat bantu refleksi diri dan tidak memberikan diagnosis medis atau interpretasi psikologis profesional.



Gambar 2.2 Diagram Konteks

2.3 Karakteristik Pengguna

Tabel 2.3.1. Karakteristik Pengguna

Kategori Pengguna	Tugas	Hak Akses ke aplikasi
Pengguna Terdaftar	Mengelola data pribadi dan histori emosional	- Membuat, membaca, mengubah, dan menghapus entri mood harian - Menambahkan dan mengedit jurnal pribadi - Melihat grafik tren mingguan/bulanan - Mencari jurnal berdasarkan tanggal/kata kunci - Mengelola target self-care pribadi - Mengelola akun (ubah password, profil) - Mengunduh atau mengeksport ringkasan data pribadi (jika fitur tersedia)

2.4 Batasan Perangkat Lunak

Untuk mengakomodir keperluan perangkat lunak, juga tetap menjaga sistem yang optimal, sekaligus memantau kesehatan mental pengguna secara terukur, ditetapkan beberapa batasan teknis sebagai berikut:

1. P/L harus menggunakan format JSON (JavaScript Object Notation) untuk menjamin efisiensi dari pertukaran data internal.
2. P/L harus mengakomodasi pencatatan log suasana hati, refleksi diri terstruktur, dan pemantauan target self-care harian secara konsisten.

3. P/L harus mengimplementasikan sistem manajemen basis data relasional yang tertanam secara lokal untuk menjaga konsistensi dan integritas data pengguna.
4. P/L dibatasi untuk tidak memberikan interpretasi medis, diagnosis, dan saran klinis terhadap kondisi kesehatan mental pengguna.
5. P/L harus dapat memvisualisasikan tren mingguan dan ringkasan bulanan dari data lokal guna mempermudah pemahaman pola bagi pengguna.
6. P/L harus menjaga data pribadi pengguna melalui implementasi enkripsi pada level basis data dan mekanisme kunci akses aplikasi untuk mencegah akses fisik yang tidak diinginkan.
7. P/L harus dirancang untuk beroperasi secara lokal, guna memastikan aksesibilitas aplikasi tanpa ketergantungan penuh pada koneksi internet.

2.5 Lingkungan Operasi Perangkat Lunak

Dengan gambaran umum dari P/L ini, diperlukannya lingkungan operasi yang sesuai dengan kebutuhan berjalannya P/L tersebut, maka dari itu diambil spesifikasi sebagai berikut,

Server : Localhost yang berbasis Python
Client : Perangkat yang dapat menjalankan aplikasi mandiri dengan dukungan pemrosesan grafis lokal
DBMS : SQLite sebagai sistem manajemen basis data relasional untuk menyimpan secara lokal tanpa perlunya proses server database yang lain
OS : Windows 10/11 dengan arsitektur 64-bit

3 Deskripsi Kebutuhan Perangkat Lunak

3.1 Kebutuhan Fungsional

Tabel 3.1.1. Kebutuhan Fungsional

ID	Kebutuhan	Penjelasan
F01	P/L mampu menyimpan data suasana hati harian pengguna	Pengguna dapat memberikan data suasana hati setiap hari yang selanjutnya disimpan dalam <i>database</i> . Pengguna memberikan input tingkat suasana hati, judul, deskripsi, foto, pemicu, tanggal dan waktu
F02	P/L mampu menyunting data suasana hati harian pengguna	Pengguna dapat menyunting data suasana hati yang sebelumnya telah disimpan di dalam <i>database</i>
F03	P/L mampu menghapus data suasana hati harian pengguna	Pengguna dapat menghapus data suasana hati yang sebelumnya telah disimpan di dalam <i>database</i>
F04	P/L dapat menyimpan jurnal pribadi yang diisi oleh pengguna	Pengguna dapat mengisi jurnal pribadi atau refleksi dalam bentuk teks yang selanjutnya disimpan dalam <i>database</i>
F05	P/L dapat menyunting jurnal pribadi yang diisi oleh pengguna	Pengguna dapat menyunting jurnal pribadi yang sebelumnya telah disimpan di dalam <i>database</i>
F06	P/L dapat menghapus jurnal pribadi yang diisi oleh pengguna	Pengguna dapat menyunting jurnal pribadi yang sebelumnya telah disimpan di dalam <i>database</i>
F07	P/L mampu mengidentifikasi pola emosional dalam periode mingguan atau bulanan melalui visualisasi sederhana	Pengguna dapat melihat hasil visualisasi dari pola emosional dalam periode mingguan atau bulanan melalui <i>homepage</i>
F08	P/L dapat melakukan pencatatan target <i>self-care</i> atau aktivitas positif	Pengguna dapat mencatat nama aktivitas dan frekuensi/durasi target <i>self-care</i> atau aktivitas positif
F09	P/L dapat menyunting target <i>self-care</i> atau aktivitas positif	Pengguna dapat menyunting target <i>self-care</i> yang sebelumnya telah disimpan di dalam <i>database</i>
F10	P/L dapat menghapus target <i>self-care</i> atau aktivitas positif	Pengguna dapat menyunting target <i>self-care</i> yang sebelumnya telah disimpan di dalam <i>database</i>
F11	P/L mampu memberikan visualisasi progres ketercapaian target <i>self-care</i> atau aktivitas positif	Pengguna dapat memantau progres kebiasaan yang mendukung kesejahteraan mental melalui hasil pencatatan target <i>self-care</i> atau aktivitas positif

F12	P/L mampu menyediakan ringkasan statistik atau grafik tren suasana hati	Pengguna dapat melihat ringkasan statistik atau grafik tren suasana hati dari hasil pengolahan <i>database</i> . Grafik dan statistik hanya bersifat informatif dan deskriptif tanpa mengandung interpretasi medis/diagnosa.
F13	P/L dapat melakukan pencarian entri jurnal berdasarkan tanggal atau kata kunci	Pengguna dapat mencari entri jurnal dari tanggal atau kata kunci

3.2 Kebutuhan Non Fungsional

Tabel 3.2.1. Tabel Kebutuhan Non Fungsional

ID	Parameter	Kebutuhan
NF01	Availability	Aplikasi harus dapat diakses minimal 99% dalam periode 1 bulan (kecuali waktu maintenance terjadwal)
NF02	Reliability	Sistem tidak boleh kehilangan data jurnal atau mood yang telah berhasil disimpan (data persistence 100% setelah notifikasi “berhasil disimpan” muncul).
NF03	Ergonomy	Antarmuka harus dapat digunakan oleh pengguna baru tanpa pelatihan khusus.
NF04	Portability	Sistem harus mendukung portabilitas sehingga dapat beroperasi dengan baik pada lingkungan sistem operasi Windows.
NF05	Memory	N/A
NF06	Response Time	Sistem harus mampu menampilkan hasil dalam 4 detik hingga 5 tahun penggunaan.
NF07	Safety	N/A
NF08	Security	N/A
NF09	Others 1: Bahasa komunikasi	Seluruh antarmuka, notifikasi, dan bantuan penggunaan harus tersedia dalam Bahasa Indonesia yang jelas dan tidak mengandung istilah medis yang bersifat diagnosis.

NF10	Storage management	Sistem harus mampu menyimpan file berupa foto yang diupload oleh user
------	--------------------	---

4 Model Analisis

4.1 Model Use Case

4.1.1 Identifikasi Actor

Tabel 4.1.1.1. Identifikasi Actor

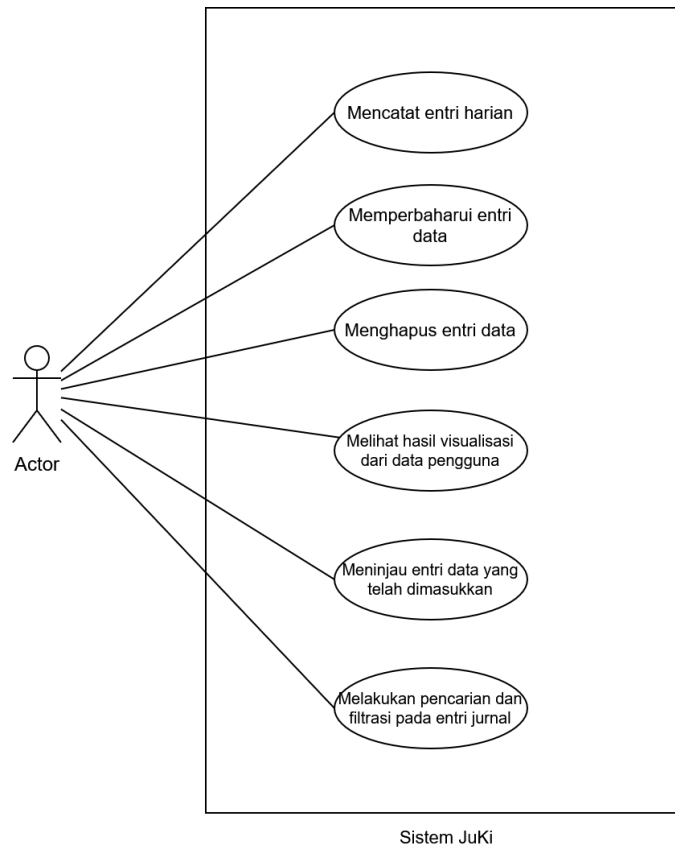
Kode Actor	Actor	Deskripsi
A01	User	<i>Actor</i> dengan role ini mempunyai wewenang sebagai berikut: - Menambahkan, mengubah, dan menghapus suasana hati harian - Menambahkan, mengubah, dan menghapus jurnal pribadi - Menambahkan, mengubah, dan menghapus target self-care/kegiatan positif - Melihat visualisasi dari pola emosional dalam periode mingguan atau bulanan - Memantau progres ketercapaian target self-care atau aktivitas positif - Melihat ringkasan statistik atau grafik tren suasana hati - Mencari entri jurnal dari tanggal atau kata kunci

4.1.2 Identifikasi Use Case

Tabel 4.1.2.1. Identifikasi Use Case

No	Use Case	Deskripsi
UC01	Mencatat entri harian	User mencatat data suasana hati harian dengan memasukkan tingkat suasana hati, judul, deskripsi refleksi, foto, pemicu emosi, tanggal, dan waktu. User juga dapat menambahkan entri jurnal serta target kegiatan self-care.
UC02	Melihat hasil visualisasi dari data pengguna	Sistem mengolah data lokal untuk membuat tren fluktuasi suasana hari dan statistik persentase ketercapaian target self-care. Hasil tersebut dapat dilihat oleh user dalam bentuk visualisasi data.
UC03	Menghapus entri data	User menghapus entri suasana hati, jurnal, dan kegiatan self-care yang telah disimpan sebelumnya.
UC04	Memperbaharui entri data	User mengubah konten entri jurnal, suasana hati, dan kegiatan self-care yang telah disimpan sebelumnya.
UC05	Meninjau entri data yang telah dimasukkan	User melihat kembali data suasana hati harian, entri jurnal pribadi, dan target kegiatan self-care yang telah tersimpan di dalam sistem.
UC6	Melakukan pencarian dan filtrasi pada entri jurnal	User mencari data pada sistem berdasarkan kategori suasana hati, tanggal, atau kata kunci pada entri jurnal.

4.1.3 Diagram Use Case



Gambar 4.1 Diagram Use Case

4.1.4 Skenario Use Case

4.1.4.1 Skenario Use Case 01

Nama Use Case: Mencatat entri harian

Skenario:

Tabel 4.1.4.1.1 Skenario Use Case 01

Aksi Actor	Reaksi Sistem
Skenario Normal	
1. Memilih menu pencatatan entri harian	
	2. Sistem menampilkan halaman form pencatatan entri
3. Memilih kategori suasana hati	
4. Mengisi entri jurnal (judul dan deskripsi)	
5. Mengisi target kegiatan self-care	
6. Menekan tombol simpan	

	7. Sistem memvalidasi data yang dimasukkan
	8. Sistem menyimpan data suasana hati, entri jurnal, dan target self-care ke penyimpanan lokal
	9. Sistem menampilkan notifikasi bahwa data berhasil disimpan
Skenario Alternatif 1 (Data tidak lengkap)	
1. Mengirim form dengan beberapa field wajib yang belum diisi	
	2. Sistem mendeteksi bahwa terdapat field wajib yang kosong
	3. Sistem menampilkan pesan bahwa data belum lengkap
	4. Sistem meminta pengguna melengkapi data sebelum menyimpan
Skenario Alternatif 2 (Data tidak valid)	
1. Mengirim form dengan data yang format atau nilainya tidak sesuai aturan sistem	
	2. Sistem melakukan validasi terhadap format data
	3. Sistem menampilkan pesan kesalahan bahwa data tidak valid
	4. Sistem meminta pengguna memperbaiki data sebelum menyimpan
Skenario Alternatif 3 (User membatalkan pencatatan)	
1. Menekan tombol batal	
	2. Sistem membatalkan proses pencatatan
	3. Sistem kembali ke halaman utama

4.1.4.2 Skenario Use Case 02

Nama Use Case: Menghasilkan visualisasi dari data pengguna

Skenario:

Tabel 4.1.4.1.2 Skenario Use Case 02

Aksi Actor	Reaksi Sistem
Skenario Normal	
1. Memilih menu visualisasi data	
	2. Sistem mengambil data suasana hati dan target self-care dari basis data
	3. Sistem mengolah data menjadi grafik atau statistik
	4. Sistem menampilkan visualisasi tren kepada pengguna
	5. Sistem menampilkan halaman utama aplikasi

Skenario Alternatif 1 (Data belum tersedia)	
1. Memilih menu visualisasi data	
	2. Sistem tidak menemukan data dalam basis data
	3. Sistem menampilkan pesan “Belum ada data untuk divisualisasikan”

4.1.4.3 Skenario Use Case 03

Nama Use Case: Menghapus entri data

Skenario:

Tabel 4.1.4.1.3 Skenario Use Case 03

Aksi Actor	Reaksi Sistem
Skenario Normal	
1. Membuka daftar entri	
	2. Sistem menampilkan daftar entri
3. Memilih entri yang ingin dihapus	
4. Menekan tombol hapus	
	5. Sistem meminta konfirmasi penghapusan
6. Mengkonfirmasi penghapusan	
	7. Sistem menghapus data dari basis data
	8. Sistem memperbarui tampilan daftar entri
Skenario Alternatif 1 (User membatalkan penghapusan)	
1. Menekan tombol batal pada konfirmasi	
	2. Sistem membatalkan proses penghapusan
	3. Sistem kembali ke daftar entri

4.1.4.4 Skenario Use Case 04

Nama Use Case: Memperbaharui entri data

Skenario:

Tabel 4.1.4.1.4 Skenario Use Case 04

Aksi Actor	Reaksi Sistem
Skenario Normal	
1. Membuka daftar entri	
	2. Sistem menampilkan daftar entri
3. Memilih entri yang ingin diperbarui	
	4. Sistem menampilkan halaman edit
5. Mengubah isi entri	
6. Menekan tombol simpan	
	7. Sistem memvalidasi perubahan data
	8. Sistem menyimpan perubahan pada basis data
	9. Sistem menampilkan notifikasi berhasil

Skenario Alternatif 1 (Data tidak lengkap)	
1. Mengirim perubahan data dengan beberapa field wajib yang belum diisi	
	2. Sistem mendeteksi bahwa terdapat field yang belum lengkap
	3. Sistem menampilkan pesan bahwa data harus dilengkapi
	4. Sistem meminta pengguna melengkapi data sebelum menyimpan perubahan
Skenario Alternatif 2 (Data tidak valid)	
1. Mengirim perubahan data dengan format atau nilai yang tidak sesuai aturan sistem	
	2. Sistem melakukan validasi data
	3. Sistem menampilkan pesan kesalahan bahwa format data tidak valid
	4. Sistem meminta pengguna memperbaiki data sebelum menyimpan perubahan

4.1.4.5 Skenario Use Case 05

Nama Use Case: Meninjau entri data yang telah dimasukkan

Skenario:

Tabel 4.1.4.1.5 Skenario Use Case 05

Aksi Actor	Reaksi Sistem
Skenario Normal	
1. Memilih menu daftar entri	
	2. Sistem mengambil data dari basis data
	3. Sistem menampilkan daftar entri
4. Memilih salah satu entri	
	5. Sistem menampilkan detail entri
Skenario Alternatif (Data tidak tersedia)	
1. Memilih menu daftar entri	
	2. Sistem tidak menemukan data
	3. Sistem menampilkan pesan “Belum ada entri yang tersedia”

4.1.4.6 Skenario Use Case 06

Nama Use Case: Melakukan pencarian dan filtrasi pada entri jurnal

Skenario:

Tabel 4.1.4.1.6 Skenario Use Case 06

Aksi Actor	Reaksi Sistem
Skenario Normal	
1. Membuka fitur pencarian jurnal	

	2. Sistem menampilkan kolom pencarian
3. Memasukkan kata kunci / memilih filter	
4. Menekan tombol cari	
	5. Sistem melakukan pencarian pada basis data
	6. Sistem menampilkan hasil pencarian
Skenario Alternatif 1 (Data tidak ditemukan)	
1. Memasukkan parameter pencarian	
	2. Sistem tidak menemukan data
	3. Sistem menampilkan pesan “Data tidak ditemukan”
Skenario Alternatif 2 (Parameter pencarian kosong)	
1. Menekan tombol cari tanpa memasukkan parameter	
	2. Sistem menampilkan pesan bahwa parameter pencarian harus diisi

5 Diagram Kelas

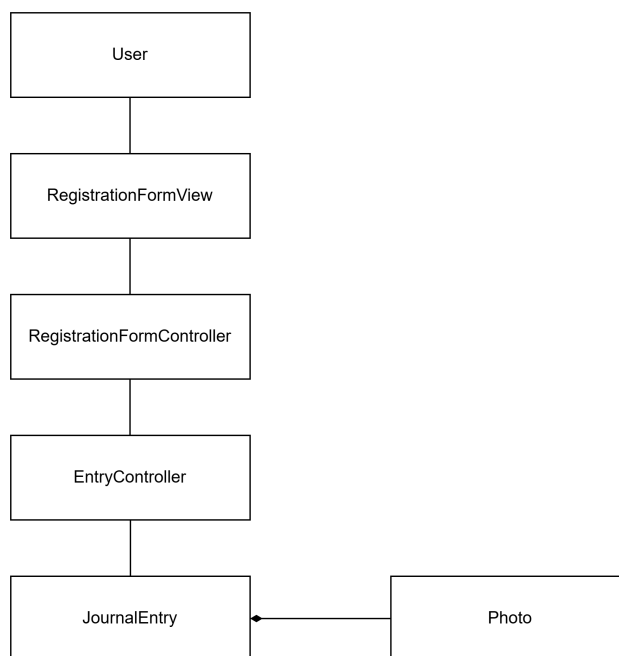
5.1 Use Case Mencatat Entri Harian

5.1.1 Identifikasi Kelas Use Case Mencatat Entri Harian

Tabel 5.1.1. Daftar Kelas Use Case Mencatat Entri Harian

ID Kelas	Nama Kelas	Deskripsi Kelas	ID Use Case
C-01	User	Merepresentasikan pengguna yang berinteraksi dengan sistem untuk melakukan input data	UC01
C-02	RegistrationFormView	Antarmuka yang menerima dan melakukan validasi terhadap masukan dari user	UC01
C-03	JournalEntry	Entitas yang menyimpan informasi entri jurnal harian	UC01
C-04	Photo	Merepresentasikan objek foto dalam sistem yang memiliki fungsi utama untuk mengelola proses unggah dan penghapusan foto.	UC01
C-07	EntryController	Mengelola alur pengambilan data entri dari Repository.	UC01
C-13	RegistrationFormController	Melakukan validasi data terhadap input user dan memberikan notifikasi informasi untuk memberi tahu kesesuaian informasi	UC01

5.1.2 Identifikasi Kelas Use Case Mencatat Entri Harian



Gambar 5.1 Use Case Mencatat Entri Harian

Tabel 5.1.2. Identifikasi Kelas Use Case Mencatat Entri Harian

ID Kelas	Nama Kelas	Atribut	Metode/Operasi
C-01	User	- name: String - id: Integer	+ fillForm(): void + pressSave(): void + pressCancel(): void + getName(): String + getId(): Integer + setName(name: String): void
C-02	RegistrationFormView	-	+showPage() : void + displayMessage(msg: String): void + selectMenu(): void
C-03	JournalEntry	- id: Integer - category: String - title: String - description: String - trigger: String - target: String - date: Date - time: Time - photo: Photo	+ createEntry(category: String, title: String, description: String, trigger: String, target: String, date: Date, time: Time, photo: Photo) + getId(): Integer + getCategory(): String + getTitle(): String + getDescription(): String + getTrigger(): String + getTarget(): String + getDate(): Date + getTime(): Time + getPhoto(): Photo + setCategory(category: String): void + setTitle(title: String): void + setDescription(description: String): void + setTrigger(trigger: String): void + setTarget(target: String): void + setDate(date: Date): void

			+ setTime(time: Time): void + setPhoto(photo: Photo): void
C-04	Photo	- id: Integer - filePath: String	+ uploadPhoto(): void + deletePhoto(): void + getId(): Integer + getFilePath(): String + setFilePath(filePath: String): void
C-07	EntryController	- journal: List<JournalEntry>	+ getAllEntries(): List<JournalEntry> + getEntryDetail(id: Integer): JournalEntry + isEmpty(): Boolean + addEntry(entry: JournalEntry): void + deleteEntry(id: Integer): void + cancelEntry(id: Integer): void + updateEntry(entry: JournalEntry): void
C-13	RegistrationFormController	- validation_status : Boolean	+ showNotification() : void + validateData() : Boolean + submitEntry(data: Object): void

5.2 Use Case Melihat Hasil Visualisasi dari Data Pengguna

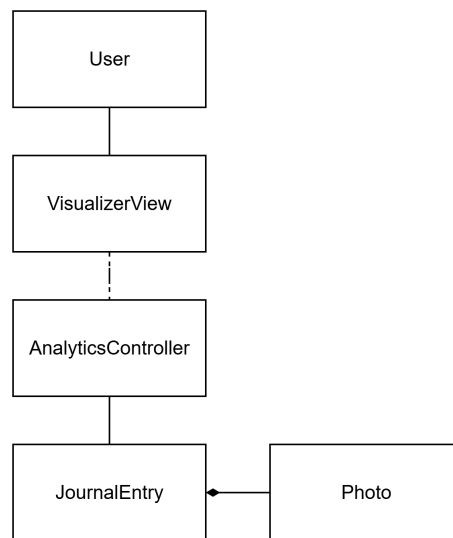
5.2.1 Identifikasi Kelas Use Case Melihat Hasil Visualisasi dari Data Pengguna

Tabel 5.2.1. Daftar Kelas Use Case Melihat Hasil Visualisasi dari Data Pengguna

ID Kelas	Nama Kelas	Deskripsi Kelas	ID Use Case
C-01	User	Merepresentasikan pengguna yang berinteraksi dengan sistem untuk melakukan input data	UC02
C-03	JournalEntry	Entitas yang menyimpan informasi entri jurnal harian	UC02

C-04	Photo	Merepresentasikan objek foto dalam sistem yang memiliki fungsi utama untuk mengelola proses unggah dan penghapusan foto.	UC02
C-05	VisualizerView	Memetakan hasil AnalyticsController untuk dijadikan visualisasi yang menggambarkan progress pengguna dari entri ke entri.	UC02
C-06	AnalyticsController	Mengolah data mentah menjadi data yang siap digunakan untuk membuat visualisasi dari semua entri yang telah dibuat oleh pengguna	UC02

5.2.2 Identifikasi Kelas Use Case Melihat Hasil Visualisasi dari Data Pengguna



Gambar 5.2 Use Case Melihat Hasil Visualisasi dari Data Pengguna

Tabel 5.2.2. Identifikasi Kelas Use Case Melihat Hasil Visualisasi dari Data Pengguna

ID Kelas	Nama Kelas	Atribut	Metode/Operasi
C-01	User	- name: String - id: Integer	+ fillForm(): void + pressSave(): void + getName(): String + getId(): Integer + setName(name: String): void
C-03	JournalEntry	- id: Integer - category: String - title: String - description: String	+ createEntry(category: String, title: String, description: String, trigger: String, target:

		- trigger: String - target: String - date: Date - time: Time - photo: Photo	String, date: Date, time: Time, photo: Photo) + getId(): Integer + getCategory(): String + getTitle(): String + getDescription(): String + getTrigger(): String + getTarget(): String + getDate(): Date + getTime(): Time + getPhoto(): Photo + setCategory(category: String): void + setTitle(title: String): void + setDescription(description: String): void + setTrigger(trigger: String): void + setTarget(target: String): void + setDate(date: Date): void + setTime(time: Time): void + setPhoto(photo: Photo): void
C-04	Photo	- id: Integer - filePath: String	+ uploadPhoto(): void + deletePhoto(): void + getId(): Integer + getFilePath(): String + setFilePath(filePath: String): void
C-05	VisualizerView	- period_filter: String - chart_type: String - summary: String - vis_width: Integer - vis_height: Integer - chart_color: String	+getPeriodFilter(): String +setPeriodFilter(String): void +getChartType(): String +setChartType(String): void +getCanvasSize(): Integer, Integer + setCanvasSize(w: Integer, h: Integer): void +getChartColor(): String

			+ applyColorTheme(hex: String): void + updateDisplay(): void + showNotification() : void + selectMenu(): void + displayVisualization():void
C-06	AnalyticsController	- raw_entries: List<JournalEntry> - success_rate: Float - mood_trend: Map - total_target_count: Integer	+calculateAverageMood(): Float +calculateSelfCareRate(): Float +filterByData(startDate, endDate): void +aggregateDate(): void +getMoodTrend(): Map +getJournalEntry():List<JournalEntry> +getVisualization:List<JournalEntry>

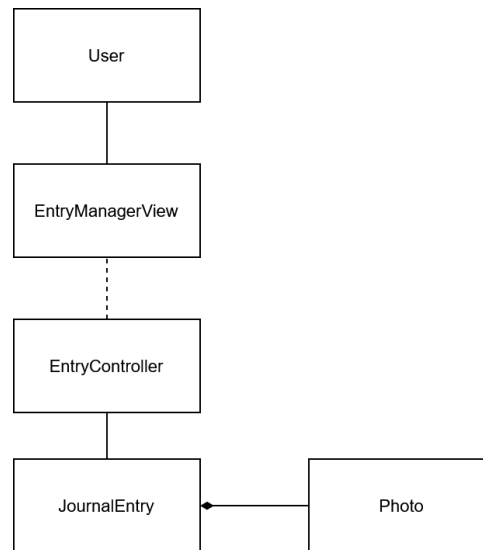
5.3 Use Case Menghapus Entri Data

5.3.1 Identifikasi Kelas Use Case Menghapus Entri Data

Tabel 5.3.1. Daftar Kelas Use Case Menghapus Entri Data

ID Kelas	Nama Kelas	Deskripsi Kelas	ID Use Case
C-01	User	Merepresentasikan pengguna yang berinteraksi dengan sistem untuk melakukan input data	UC03
C-03	JournalEntry	Merepresentasikan entri jurnal yang tersimpan dalam basis data lokal	UC03
C-04	Photo	Merepresentasikan objek foto dalam sistem yang memiliki fungsi utama untuk mengelola proses unggah dan penghapusan foto.	UC03
C-07	EntryController	Mengelola alur pengambilan data entri dari Repository.	UC03
C-08	EntryManagerView	Merepresentasikan antarmuka yang menampilkan opsi mengedit dan menghapus entri.	UC03

5.3.2 Identifikasi Kelas Use Case Menghapus Entri Data



Gambar 5.3. Class Diagram Menghapus Entri Data

Tabel 5.3.2. Spesifikasi Kelas Use Case Menghapus Entri Data

ID Kelas	Nama Kelas	Atribut	Metode/Operasi
C-01	User	- name: String - id: Integer	+ fillForm(): void + pressSave(): void + getName(): String + getId(): Integer + setName(name: String): void
C-03	JournalEntry	- id: Integer - category: String - title: String - description: String - trigger: String - target: String - date: Date - time: Time - photo: Photo	+ createEntry(category: String, title: String, description: String, trigger: String, target: String, date: Date, time: Time, photo: Photo) + getId(): Integer + getCategory(): String + getTitle(): String + getDescription(): String + getTrigger(): String + getTarget(): String + getDate(): Date + getTime(): Time

			+ getPhoto(): Photo + setCategory(category: String): void + setTitle(title: String): void + setDescription(description: String): void + setTrigger(trigger: String): void + setTarget(target: String): void + setDate(date: Date): void + setTime(time: Time): void + setPhoto(photo: Photo): void
C-04	Photo	- id: Integer - filePath: String	+ uploadPhoto(): void + deletePhoto(): void + getId(): Integer + getFilePath(): String + setFilePath(filePath: String): void
C-07	EntryController	- journal: List<JournalEntry>	+ getAllEntries(): List<JournalEntry> + getEntryDetail(id: Integer): JournalEntry + isEmpty(): Boolean + addEntry(entry: JournalEntry): void + deleteEntry(id: Integer): void + updateEntry(entry: JournalEntry): void
C-08	EntryManagerView	controller: EntryController	+ displayEntryOptions(entry: JournalEntry): void + confirmDelete(): Boolean + getUserAction(): String + selectMenu(): void

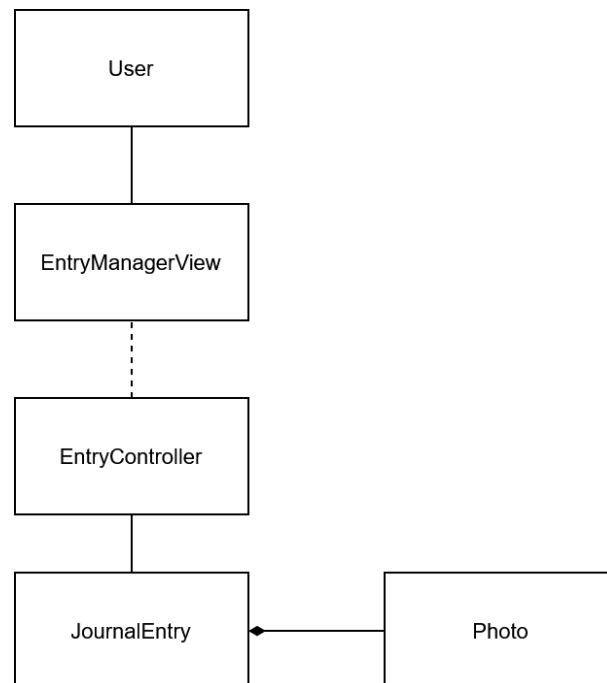
5.4 Use Case Memperbaharui Entri Data

5.4.1 Identifikasi Kelas Use Case Memperbaharui Entri Data

Tabel 5.4.1. Daftar Kelas Use Case Memperbaharui Entri Data

ID Kelas	Nama Kelas	Deskripsi Kelas	ID Use Case
C-01	User	Merepresentasikan pengguna yang berinteraksi dengan sistem untuk melakukan input data	UC04
C-03	JournalEntry	Merepresentasikan entri jurnal yang tersimpan dalam basis data lokal	UC04
C-04	Photo	Merepresentasikan objek foto dalam sistem yang memiliki fungsi utama untuk mengelola proses unggah dan penghapusan foto.	UC04
C-07	EntryController	Mengelola alur pengambilan data entri dari Repository.	UC04
C-08	EntryManagerView	Merepresentasikan antarmuka yang menampilkan opsi mengedit dan menghapus entri.	UC04

5.4.2 Identifikasi Kelas Use Case Memperbaharui Entri Data



Gambar 5.4. Class Diagram Memperbaharui Entri Data

Tabel 5.4.2. Spesifikasi Kelas Use Case Memperbaharui Entri Data

ID Kelas	Nama Kelas	Atribut	Metode/Operasi
C-01	User	- name: String - id: Integer	+ fillForm(): void + pressSave(): void + getName(): String + getId(): Integer + setName(name: String): void
C-03	JournalEntry	- id: Integer - category: String - title: String - description: String - trigger: String - target: String - date: Date - time: Time - photo: Photo	+ createEntry(category: String, title: String, description: String, trigger: String, target: String, date: Date, time: Time, photo: Photo) + getId(): Integer + getCategory(): String + getTitle(): String + getDescription(): String + getTrigger(): String + getTarget(): String + getDate(): Date + getTime(): Time + getPhoto(): Photo + setCategory(category: String): void + setTitle(title: String): void + setDescription(description: String): void + setTrigger(trigger: String): void + setTarget(target: String): void + setDate(date: Date): void + setTime(time: Time): void + setPhoto(photo: Photo): void
C-04	Photo	- id: Integer - filePath: String	+ uploadPhoto(): void + deletePhoto(): void + getId(): Integer + getFilePath(): String

			+ setFilePath(filePath: String): void
C-07	EntryController	- journal: List<JournalEntry>	+ getAllEntries(): List<JournalEntry> + getEntryDetail(id: Integer): JournalEntry + isEmpty(): Boolean + addEntry(entry: JournalEntry): void + deleteEntry(id: Integer): void + updateEntry(entry: JournalEntry): void
C-06	EntryManagerView	controller: EntryController	+displayEntryOptions(entry : JournalEntry): void + confirmDelete(): Boolean + getUserAction(): String + selectMenu(): void

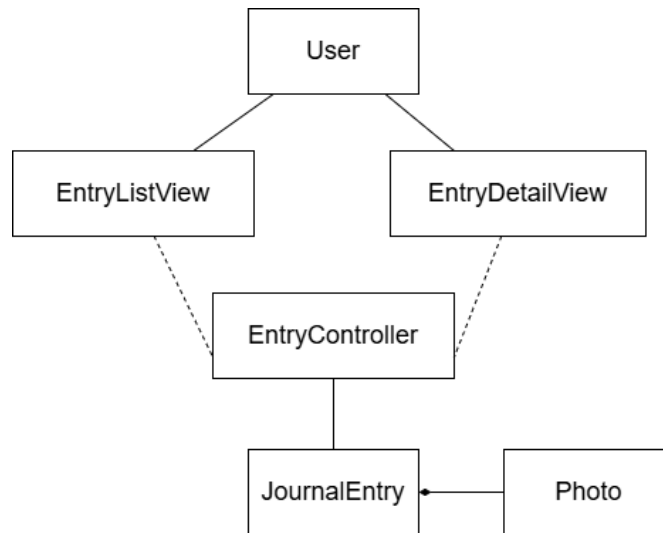
5.5 Use Case Meninjau Entri Data yang Telah Dimasukkan

5.5.1 Identifikasi Kelas Use Case Meninjau Entri Data yang Telah Dimasukkan

Tabel 5.5.1. Daftar Kelas Berdasarkan Use Case Meninjau Entri Data yang Telah Dimasukkan

ID Kelas	Nama Kelas	Deskripsi Kelas	ID Use Case
C-01	User	Merepresentasikan pengguna yang berinteraksi dengan sistem untuk melakukan input data.	UC05
C-03	JournalEntry	Entitas yang menyimpan informasi entri jurnal harian.	UC05
C-04	Photo	Merepresentasikan objek foto dalam sistem yang memiliki fungsi utama untuk mengelola proses unggah dan penghapusan foto.	UC05
C-09	EntryListView	Merepresentasikan antarmuka yang menampilkan daftar entri jurnal yang ingin diakses kepada pengguna.	UC05
C-10	EntryDetailView	Merepresentasikan antarmuka yang menampilkan informasi lengkap dari entri jurnal yang dipilih oleh pengguna.	UC05
C-07	EntryController	Mengelola alur pengambilan data entri dari Repository .	UC05

5.5.2 Identifikasi Kelas Use Case Meninjau Entri Data yang Telah Dimasukkan



Gambar 5.5 Meninjau Entri Data yang Telah Dimasukkan

Tabel 5.5.2. Spesifikasi Kelas Use Case Meninjau Entri Data yang Telah Dimasukkan

ID Kelas	Nama Kelas	Atribut	Metode/Operasi
C-01	User	- name: String - id: Integer	+ fillForm(): void + pressSave(): void + getName(): String + getId(): Integer + setName(name: String): void
C-03	JournalEntry	- id: Integer - category: String - title: String - description: String - trigger: String - target: String - date: Date - time: Time - photo: Photo	+ createEntry(category: String, title: String, description: String, trigger: String, target: String, date: Date, time: Time, photo: Photo) + getId(): Integer + getCategory(): String + getTitle(): String + getDescription(): String + getTrigger(): String + getTarget(): String + getDate(): Date + getTime(): Time + getPhoto(): Photo + setCategory(category: String): void

			+ setTitle(title: String): void + setDescription(description: String): void + setTrigger(trigger: String): void + setTarget(target: String): void + setDate(date: Date): void + setTime(time: Time): void + setPhoto(photo: Photo): void
C-04	Photo	- id: Integer - filePath: String	+ uploadPhoto(): void + deletePhoto(): void + getId(): Integer + getFilePath(): String + setFilePath(filePath: String): void
C-07	EntryController	- journal: List<JournalEntry>	+ getAllEntries(): List<JournalEntry> + getEntryDetail(id: Integer): JournalEntry + isEmpty(): Boolean + addEntry(entry: JournalEntry): void + deleteEntry(id: Integer): void + updateEntry(entry: JournalEntry): void
C-09	EntryListView	# controller: EntryController	+ displayEntryList(entries: List<JournalEntry>): void + displayEmptyMessage(): void + selectEntry(id: Integer): void + selectMenu(): void
C-10	EntryDetailView	# controller: EntryController	+ displayEntryDetail(entry: JournalEntry): void + returnToList(): void

5.6 Use Case Melakukan Pencarian dan Filtrasi pada Entri Jurnal

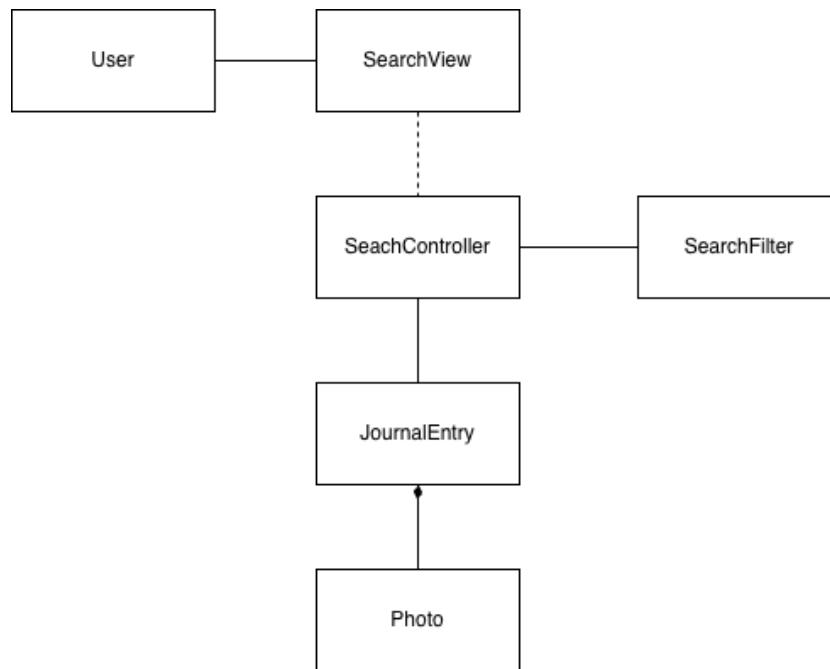
5.6.1 Identifikasi Kelas Use Case Melakukan Pencarian dan Filtrasi pada Entri Jurnal

Tabel 5.6.1. Daftar Kelas Use Case Melakukan Pencarian dan Filtrasi pada Entri Jurnal

ID Kelas	Nama Kelas	Deskripsi Kelas	ID Use Case
C-01	User	Merepresentasikan pengguna yang berinteraksi dengan sistem untuk melakukan input data	UC06
C-03	JournalEntry	Merepresentasikan entri jurnal yang tersimpan dalam basis data lokal	UC06

C-04	Photo	Merepresentasikan objek foto dalam sistem yang memiliki fungsi utama untuk mengelola proses unggah dan penghapusan foto.	UC06
C-11	SearchController	Mengelola alur pencarian jurnal dari input pengguna hingga menampilkan hasil	UC06
C-12	SearchFilter	Menyimpan dan memvalidasi parameter pencarian (kata kunci, tanggal, kategori)	UC06
C-14	SearchView	Merepresentasikan antarmuka pengguna untuk fitur pencarian, yang mengelola tampilan form pencarian, hasil pencarian, dan pesan respons sistem.	UC06

5.6.2 Identifikasi Kelas Use Case Melakukan Pencarian dan Filtrasi pada Entri Jurnal



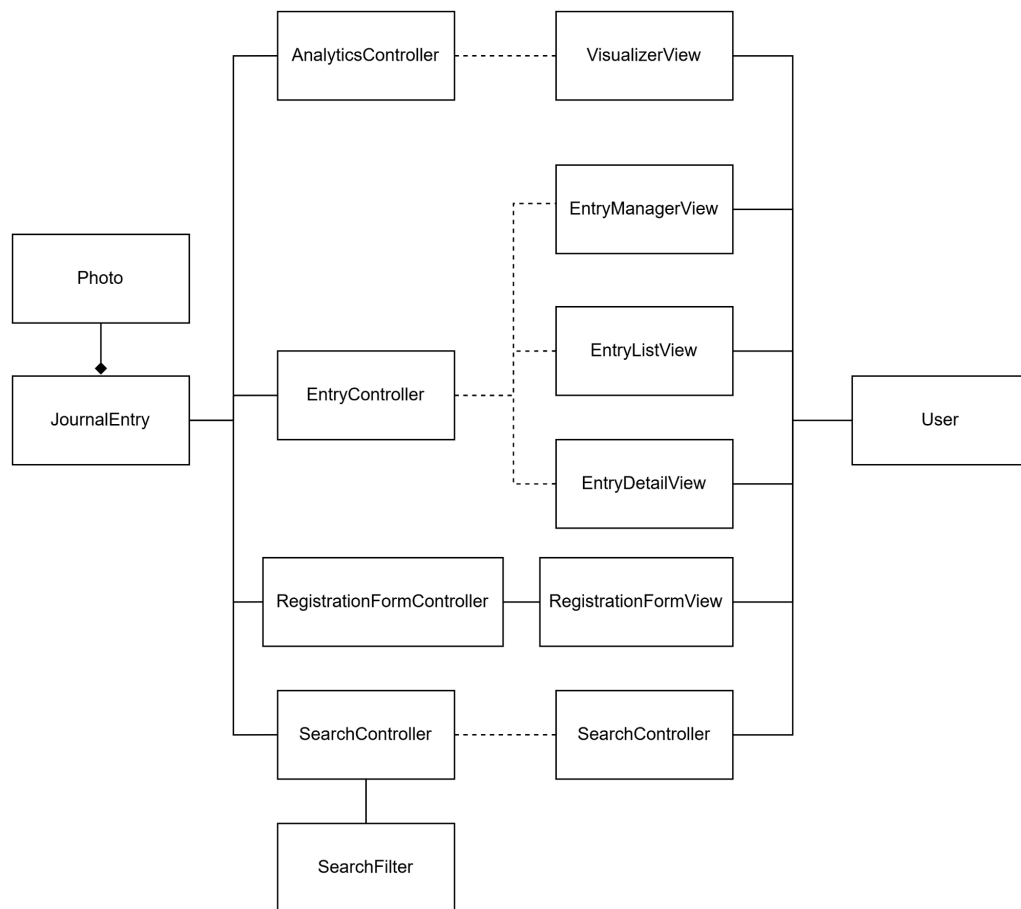
Gambar 5.6 Melakukan Pencarian dan filtrasi pada Entri Jurnal

Tabel 5.6.2. Spesifikasi Kelas Use Case Melakukan Pencarian dan Filtrasi pada Entri Jurnal

ID Kelas	Nama Kelas	Atribut	Metode/Operasi
C-01	User	- name: String - id: Integer	+ fillForm(): void + pressSave(): void + getName(): String + getId(): Integer + setName(name: String): void
C-03	JournalEntry	-id: Integer -category: String -title: String -description: String -trigger: String -target: String -date: Date -time: Time -photo: Photo	+ createEntry(category: String, title: String, description: String, trigger: String, target: String, date: Date, time: Time, photo: Photo) + getId(): Integer + getCategory(): String + getTitle(): String + getDescription(): String + getTrigger(): String

			+ getTarget(): String + getDate(): Date + getTime(): Time + getPhoto(): Photo + setCategory(category: String): void + setTitle(title: String): void + setDescription(description: String): void + setTrigger(trigger: String): void + setTarget(target: String): void + setDate(date: Date): void + setTime(time: Time): void + setPhoto(photo: Photo): void
C-04	Photo	-id: Integer -filePath: String	+ uploadPhoto(): void + deletePhoto(): void + getId(): Integer + getFilePath(): String + setFilePath(filePath: String): void
C-11	SearchController	-filter: SearchFilter	+ searchEntries(filter: SearchFilter): List<JournalEntry> + validateSearchInput(filter: SearchFilter): Boolean
C-12	SearchFilter	-keyword: String -date: Date -category : String	+ isValidFilter(): Boolean + applyFilter(): void
C-14	SearchView	-controller: SearchController	+ displaySearchForm(): void + getSearchInput(): SearchFilter + displaySearchResult(results: List<JournalEntry>): void + displayEmptyResultMessage(): void + showErrorMessage(message: String): void + selectMenu(): void

5.7 Diagram Kelas Keseluruhan



Gambar 5.7 Diagram Kelas Keseluruhan

Tabel 5.7. Spesifikasi Kelas Keseluruhan

ID Kelas	Nama Kelas	Atribut	Metode/Operasi
C-01	User	- name: String - id: Integer	+ selectMenu(): void + fillForm(): void + pressSave(): void + getName(): String + getId(): Integer + setName(name: String): void + pressCancel(): void

C-02	RegistrationFormView	-	+showPage() : void + displayMessage(msg: String): void
C-03	JournalEntry	- id: Integer - category: String - title: String - description: String - trigger: String - target: String - date: Date - time: Time - photo: Photo	+ createEntry(category: String, title: String, description: String, trigger: String, target: String, date: Date, time: Time, photo: Photo) + getId(): Integer + getCategory(): String + getTitle(): String + getDescription(): String + getTrigger(): String + getTarget(): String + getDate(): Date + getTime(): Time + getPhoto(): Photo + setCategory(category: String): void + setTitle(title: String): void + setDescription(description: String): void + setTrigger(trigger: String): void + setTarget(target: String): void + setDate(date: Date): void + setTime(time: Time): void + setPhoto(photo: Photo): void
C-04	Photo	- id: Integer - filePath: String	+ uploadPhoto(): void + deletePhoto(): void + getId(): Integer + getFilePath(): String + setFilePath(filePath: String): void
C-05	VisualizerView	- period_filter: String - chart_type: String - summary: String	+getPeriodFilter(): String +setPeriodFilter(String): void

		- vis_width: Integer - vis_height: Integer - chart_color: String	+getChartType(): String +setChartType(String): void +getCanvasSize(): Integer, Integer + setCanvasSize(w: Integer, h: Integer): void +getChartColor(): String + applyColorTheme(hex: String): void + updateDisplay(): void + showNotification() : void + selectMenu(): void +displayVisualization():void
C-06	AnalyticsController	- raw_entries: List<JournalEntry> - success_rate: Float - mood_trend: Map - total_target_count: Integer	+calculateAverageMood(): Float +calculateSelfCareRate(): Float +filterByData(startDate, endDate): void +aggregateDate(): void +getMoodTrend(): Map +getJournalEntry():List<JournalEntry> +getVisualization:List<JournalEntry>
C-07	EntryController	- journal: List<JournalEntry>	+ getAllEntries(): List<JournalEntry> + getEntryDetail(id: Integer): JournalEntry + isEmpty(): Boolean + addEntry(entry: JournalEntry): void + deleteEntry(id: Integer): void + updateEntry(entry: JournalEntry): void + cancelEntry(id: Integer): void

C-08	EntryManagerView	controller: EntryController	+displayEntryOptions(entry: JournalEntry): void + confirmDelete(): Boolean + getUserAction(): String + displayEntryList(entries: List<JournalEntry>): void + displayEntryDetail(entry: JournalEntry): void + selectMenu(): void
C-09	EntryListView	# controller: EntryController	+displayEntryList(entries: List<JournalEntry>): void +displayEmptyMessage(): void + selectEntry(id: Integer): void
C-10	EntryDetailView	# controller: EntryController	+displayEntryDetail(entry: JournalEntry): void + returnToList(): void
C-11	SearchController	-filter: SearchFilter	+ searchEntries(filter: SearchFilter): List<JournalEntry> +validateSearchInput(filter: SearchFilter): Boolean
C-12	SearchFilter	-keyword: String -date: Date -category : String	+ isValidFilter(): Boolean + applyFilter(): void
C-13	RegistrationFormController	- validation_status : Boolean	+ showNotification() void + validateData() : Boolean + submitEntry(data: Object): void
C-14	SearchView	-controller: SearchController	+ displaySearchForm(): void

			+ getSearchInput(): SearchFilter +displaySearchResult(results:List<JournalEntry>): void +displayEmptyResultMessage(): void +showErrorMessage(message: String): void
--	--	--	--

6 Keruntutan (Traceability)

6.1.1 Kebutuhan Fungsional vs Use Case

Tabel 6.1.1. Kebutuhan Fungsional vs Use Case

ID Kebutuhan Fungsional	ID Use Case Terkait
F01	UC01, UC05
F02	UC04
F03	UC03
F04	UC01, UC05
F05	UC04
F06	UC03
F07	UC02
F08	UC01, UC05
F09	UC04
F10	UC03
F11	UC02
F12	UC02, UC05
F13	UC06

6.1.2 Kebutuhan Fungsional vs Use Case vs Diagram Kelas

Tabel 6.1.2. Kebutuhan Fungsional vs Use Case vs Diagram Kelas

ID Kebutuhan Fungsional	ID Use Case Terkait	ID Class
F01	UC01, UC05	C-01, C-02, C-03, C-04, C-07, C-08, C-09, C-10
F02	UC04	C-01, C-03, C-04, C-07, C-08
F03	UC03	C-01, C-03, C-04, C-07, C-08
F04	UC01, UC05	C-01, C-02, C-03, C-04, C-07, C-08, C-09, C-10
F05	UC04	C-01, C-03, C-04, C-07, C-08
F06	UC03	C-01, C-03, C-04, C-07, C-08
F07	UC02	C-01, C-03, C-04, C-05, C-06

F08	UC01, UC05	C-01, C-02, C-03, C-04, C-07, C-08, C-09, C-10
F09	UC04	C-01, C-03, C-04, C-07, C-08
F10	UC03	C-01, C-03, C-04, C-07, C-08
F11	UC02	C-01, C-03, C-04, C-05, C-06
F12	UC02, UC05	C-01, C-03, C-04, C-05, C-06, C-07, C-09, C-10
F13	UC06	C-01, C-03, C-04, C-11, C-12, C-14